TECHNICAL ARCHITECTURE · V0.1.0 · 2026-11-09

JARVIS

A local-first AI agent for Windows —
how it is built today, layer by layer, with what works and what does not yet.

SOURCE: MAIN @ D6DF6B1 + WORKING TREE · 28 SLIDES

ELECTRON 34 · REACT 19 · TYPESCRIPT 5.7

CONTENTS

Reading map

From the outside in: context, runtime, intelligence, safety, subsystems, then quality and gaps.

I · Overview

- Jarvis today, at a glance 03
- Technology stack 04
- System context 05

II · Runtime

- Electron process model 06
- Source map (lines of code) 07
- IPC surface 08
- Request lifecycle (sequence) 09
- Agent loop internals 10
- System prompt assembly 11

III · Intelligence

- Model gateway architecture 12
- Providers & routing matrix 13
- Failover & cooldown 14
- Tool catalog 15

IV · Safety

- Permission model (ADR-004) 16
- Approval flow 17
- Gating coverage today 18

V · Subsystems

- Storage & state 19
- Browser automation (CDP) 20
- Processes & dev servers 21
- UI: four surfaces 22
- Skills: runtime vs development 23

VI · Quality & delivery

- Unit test suites 24
- MVP acceptance scoreboard 25
- Build, packaging, launch 26
- Known gaps & risks 27
- Phases & what comes next 28

I · OVERVIEW

Jarvis today, at a glance

A desktop agent: chat in, plan, call local tools through a policy layer, report back.

26	16	45	≈20k	124/124	12/20
AGENT TOOLS	PROVIDER IDS	IPC HANDLERS	LINES TS / TSX	UNIT TESTS PASS	ACCEPTANCE PASS

What it does

- Chat → model picks tools → Jarvis executes → model writes the answer.
- Files, PDFs, web search, downloads, dev servers, Chrome control, email, screen capture, memory.
- Shows a live timeline; risky steps should wait for approval.

Where it runs

- One Windows PC, one Electron app, four windows + tray icon.
- Models: cloud APIs (13 keyed/keyless) or local Ollama.
- State on disk: SQLite (sql.js), DPAPI key vault, `USER.md` memory.

Not there yet

- Approval covers 1 of 6 risky actions (slide 18).
- No task cancellation; agent loop capped at 5 turns.
- Replies arrive whole, not streamed, in the agent path.

Technology stack

Mainstream desktop web stack. No agent framework and no provider SDKs — integrations are hand-written.

LAYER	TECHNOLOGY	ROLE IN JARVIS
Runtime	Electron 34.2 · Node.js 24.13 · Chromium	Desktop shell; main process runs agent, tools, gateway
Language	TypeScript 5.7 (strict) · ES modules	Everything: main, preload, renderer, shared types
Build	electron-vite 3 (Vite 6 + esbuild) · electron-builder 25	Three bundles → `out/`; NSIS + portable x64 installers
UI	React 19 · Tailwind CSS 3.4 · Radix UI · lucide-react	Chat window, quick chat, settings app, dialogs
Content	react-markdown 10 + remark-gfm	Renders model answers (tables, code, lists)
3D	three.js 0.186 · GLB owl rig	Floating mascot with state animations
Data	sql.js 1.15 (SQLite in WASM) · Electron safeStorage (DPAPI)	Chats, tasks, events, settings · encrypted API keys
Validation	zod 3	Schema for every tool's arguments
Integrations	native `fetch` · WebSocket · `node:tls` · `child_process` · PowerShell	LLM APIs · Chrome CDP · SMTP · dev servers · SendKeys
Testing	Vitest 3 · Playwright 1.64 · axe-core	Unit + acceptance suites · UI screenshots + a11y
Typography	Fraunces · Instrument Sans · JetBrains Mono (bundled)	"Parchment & Pine" design system, no CDN (ADR-005)

System context

Everything trusted lives in one Node process. Everything else is reached over a narrow, named channel.

User
keyboard · voice later

Main window
chat, timeline, canvas

Quick chat
400×560 drawer

3D mascot
click-through overlay

Settings app
keys, models, memory

Tray icon
quick actions, status

⇄

Main process (Node, trusted)
src/main · single process · owns all side effects

- tool-calling loop
- 26 tools · zod
- ADR-004 · HMAC
- adapter · balancer
- owned child procs
- sql.js · vault · USER.md
- SKILL.md loader
- tray · mascot · settings

Renderers reach it only through `window.jarvisAPI` (preload) → 45 IPC handlers

⇄

Cloud LLM APIs
Gemini, Groq, DeepSeek, OpenAI, Anthropic, OpenRouter, NVIDIA, Mistral, Cohere, Cloudflare, Cerebras · keyless: Pollinations, OVH

Local models
Ollama · FreeLLMAPI `:31415` (legacy)

Chrome
launch with profile · CDP `127.0.0.1:9222`

Windows
files · child processes · PowerShell SendKeys · Explorer

Internet
DuckDuckGo HTML search · downloads · SMTP `:465`

Optional inbound
Local OpenAI-compatible server `127.0.0.1:31415` for IDEs (off by default)

Electron process model

Privilege stops at one boundary: renderers are sandboxed web pages; only the main process touches the OS.

Renderer × 4
one Vite bundle, surface chosen at boot

`sandbox: true` `contextIsolation` `no nodeIntegration` `strict CSP`

React UI only. No file system, no network to providers, no process control.

▼ ▲

Preload bridge
src/preload · 285 lines

`contextBridge` `window.jarvisAPI`

Typed allow-list of calls. Renderer cannot reach `ipcRenderer` directly.

▼ ▲

IPC
invoke / handle · push events

`45 ipcMain.handle` `isAuthorizedSender()`

Push: `chat:stream`, `task:updated`, `timeline:event`, `process:updated`

▼ ▲

Main process
src/main · Node 24

`agent` `tools` `policy` `gateway` `storage` `windows`

Navigation guards: `will-navigate`, `setWindowOpenHandler`.

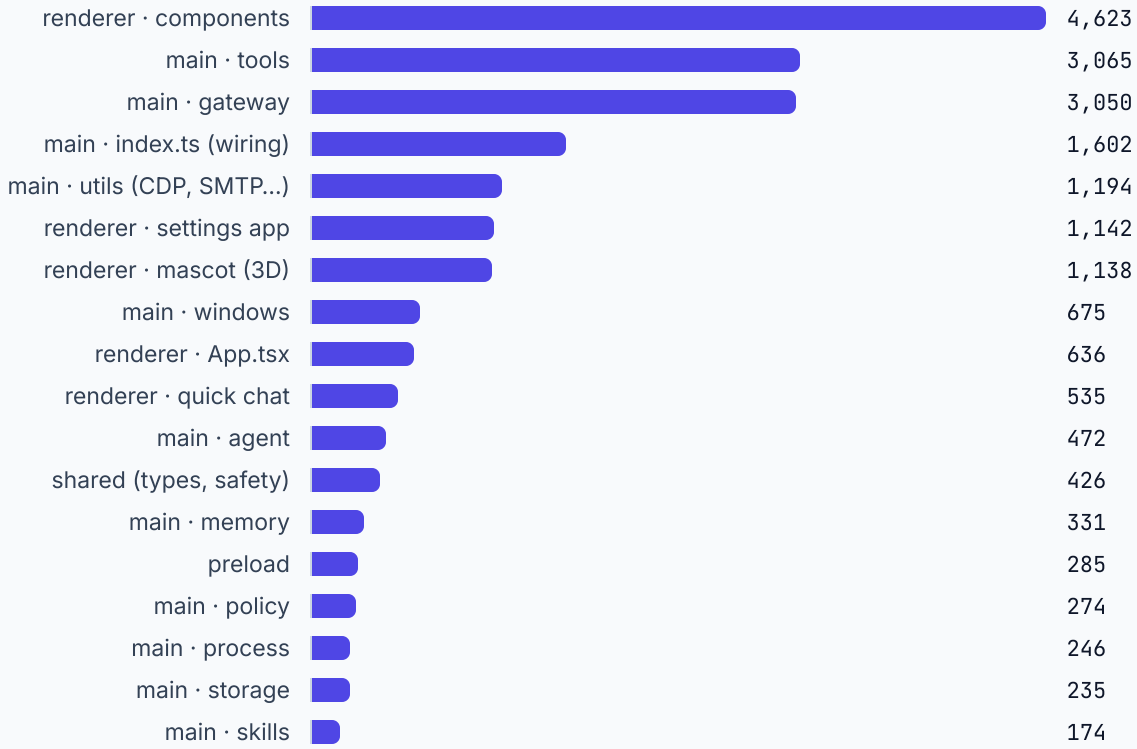
WINDOW	SPEC
Main	1200×800 · native Win11 titlebar overlay · minimize-to-tray
Quick chat	400×560 · resizable 340×440 → 900×1000
Mascot	transparent · click-through · always on top · draggable
Settings	940×700 · own window · opened by tray, header, `--settings`
Tray	not a window: `TrayManager` menu + gateway status

Wiring hub: `src/main/index.ts` (1,602 lines) creates every singleton, every window and all 45 IPC handlers. It is the one file every feature touches.

Source map — where the code lives

≈20,100 lines of TypeScript/TSX in `src/`, by module.

Lines of code by module



Concentration

Three files hold 22% of all code: `tool-executor.ts` 1,755 · `index.ts` 1,602 · `SettingsApp.tsx` 1,142. Next: `tool-registry.ts` 1,036, `model-adapter.ts` 844.

Shape

The agent core is small (agent 472 + policy 274 + process 246). Bulk sits in the UI, the tool switchboard and the gateway — the parts that grow with each new provider or tool.

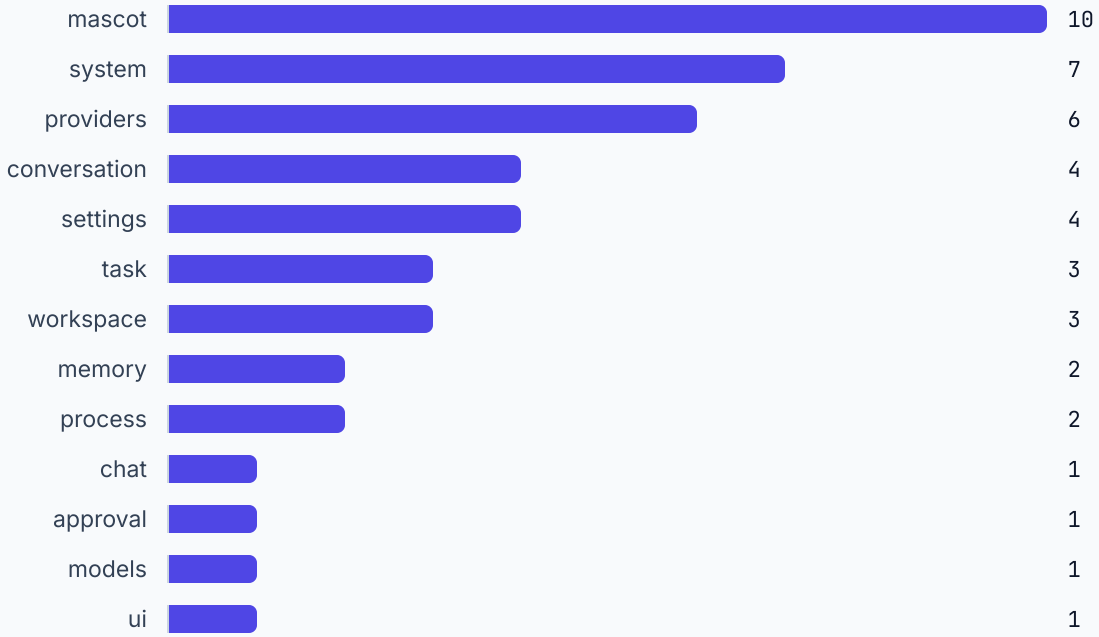
Layout

```
src/main/{agent,gateway,tools,policy,
 process,memory,skills,storage,utils,windows}
src/preload · src/shared
src/renderer/src/{components,mascot,
 quickchat,settings,lib}
```

IPC surface

45 request/response handlers, named `namespace:verb`, plus four main → renderer push channels.

Handlers per namespace



The calls that matter

- `chat:send` — runs the whole agent loop for one message.
- `task:submit-goal` — older plan-then-steps pipeline (second execution path).
- `approval:resolve` — user grants/denies a gated action.
- `task:cancel` — sets a status flag only; does not stop the loop.

Push channels (main → renderer)

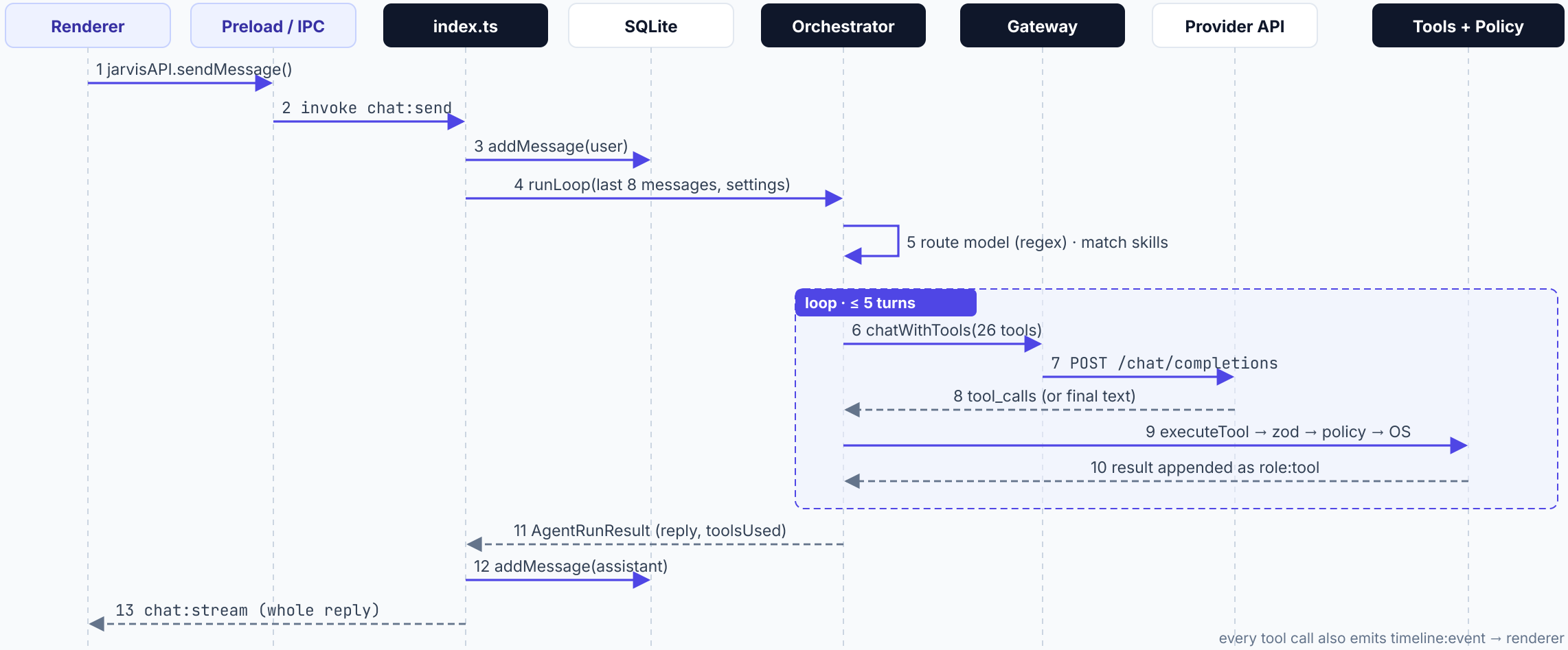
`chat:stream` · `task:updated` · `timeline:event` · `process:updated`

Guard

Sensitive handlers check `isAuthorizedSender(event.sender)` so only Jarvis's own windows can call them (SEC-005/011).

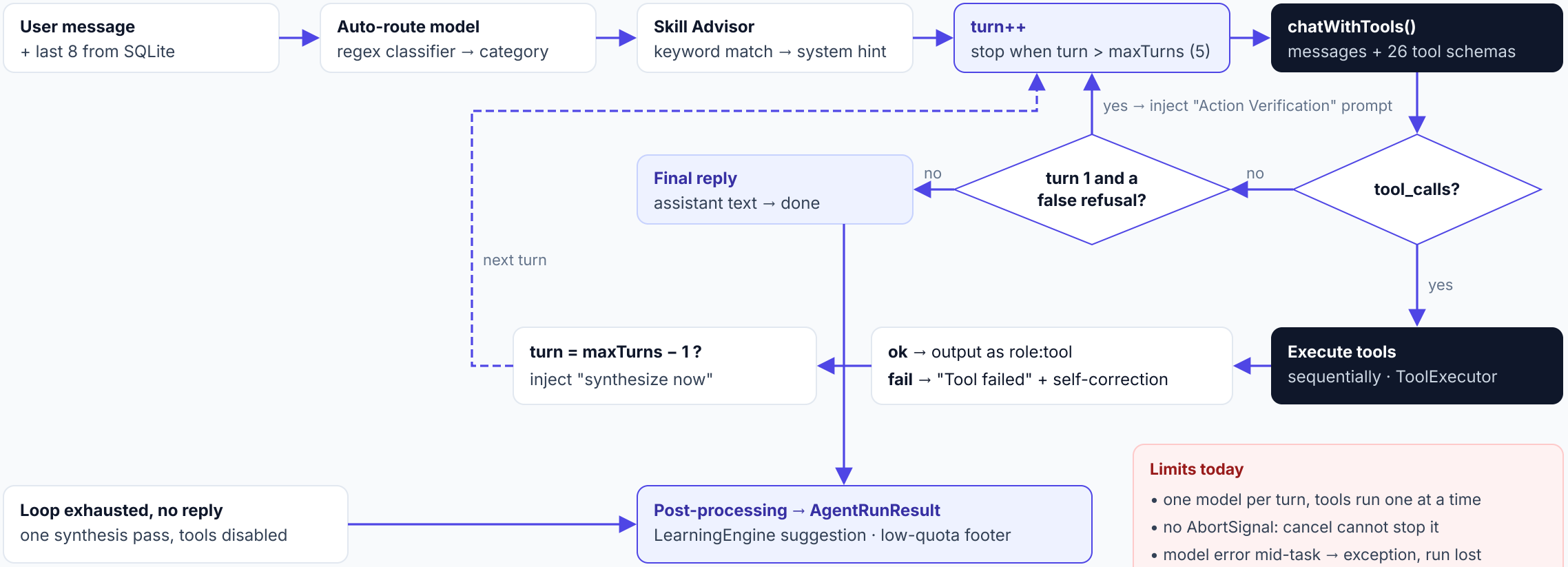
Request lifecycle — one chat message, end to end

Context is the last 8 stored messages. The reply is sent whole at the end; tool steps stream as timeline events.



Agent loop internals

`AgentOrchestrator.runLoop()` — a single-model, sequential tool-calling loop (472 lines).



Limits today

- one model per turn, tools run one at a time
- no AbortSignal: cancel cannot stop it
- model error mid-task → exception, run lost
- 5 turns: too few for search→read→write→verify
- no plan / memory of past attempts across runs

System prompt assembly

`PromptEngine.buildSystemPrompt()` composes one system message per model call from fixed and user-controlled layers.

1 · Identity & safety anchors	Who Jarvis is; immutable rules that later layers cannot override.
2 · Privacy statement	Data stays local; no technical leakage in replies.
3 · Tool usage guide	Per-tool rules: Chrome profile must be asked, Gmail flow, shortcuts, intent classification (question vs task).
4 · Style · mode · depth	`direct | balanced | detailed` `chat | debug | build | plan` `quick | standard | deep`
5 · USER.md memory	Profile, dev defaults, standing notes — injected every call.
6 · Custom prompt	User text from Settings, scanned by `scanPromptSafety()`; risky text needs an explicit acknowledgement.

Plus, per run

- **Skill Advisor** system hint when a skill matches the goal.
- **Action Verification** user message when the model falsely refuses on turn 1.
- **Synthesis nudge** at turn 4 of 5.

Prompt-safety scanner

`src/shared/prompt-safety.ts` flags bypass phrases, jailbreak patterns, destructive commands (`rmdir /s`, `format c:`, `drop database`) and credential exfiltration in the **user's own** custom prompt.

Gap

Text coming back from tools (web pages, files) is **not** scanned or labelled as untrusted before the model reads it (acceptance F).

Model gateway architecture

Embedded multi-provider engine (Phase 7). FreeLLMAPI is now a legacy fallback, not a requirement.

AgentOrchestrator
calls `chatWithTools(messages, tools, options)`

▼

ModelGatewayAdapter · model-adapter.ts (844)
adds system prompt · if the balancer throws → legacy FreeLLMAPI `127.0.0.1:31415/v1`

▼

ModelBalancer · model-balancer.ts (583)
`resolveModelForTask()` `cooldown map` `getFailoverQueue()` `listAllModels()`

▼

Provider adapters · providers/
Dedicated: `Gemini` `Groq` `DeepSeek` `OpenAI` `Anthropic` `OpenRouter` `Ollama`
Via `BaseOpenAICompatibleProvider`: `NVIDIA` `Mistral` `Cohere` `Cloudflare` `Cerebras` `Pollinations` `OVH` `custom`

▼

HTTPS
native fetch · OpenAI-style `/chat/completions` or provider dialect

CredentialVault · credential-vault.ts (476)
Keys encrypted with Electron `safeStorage` (Windows DPAPI) in `userData/provider_vault.json`. Renderer only sees `hasApiKey` + masked values. Importer migrated 7 keys from FreeLLMAPI's AES-256-GCM database.

LocalGatewayServer · local-gateway-server.ts (185)
`node:http` on `127.0.0.1:31415` exposing `/v1/models` and `/v1/chat/completions` so IDEs can reuse Jarvis's keys. Off by default.

Watch
When on: **no auth** and `Access-Control-Allow-Origin: *` — any web page can spend the keys. Same port as legacy FreeLLMAPI, so only one can run.

Providers & routing matrix

A regex classifier puts each prompt in one of four categories; the first ready provider in that row wins.

CATEGORY	TRIGGERED BY	PREFERENCE ORDER (FIRST PROVIDER WITH A KEY AND NO COOLDOWN)
coding	code fences, braces, code/bug/refactor/npm/git…	`deepseek-chat` `claude-3-5-sonnet-20241022` `gpt-4o` `gemini-2.0-flash` `llama-3.3-70b (Groq)`
reasoning	why, compare, analyze, strategy, decide…	`deepseek-reasoner` `claude-3-7-sonnet-20250219` `gemini-1.5-pro` `o3-mini`
fast_chat	short (< 40 chars) + hi/thanks/status/ping…	`llama-3.1-8b-instant (Groq)` `gemini-2.0-flash-lite` `gpt-4o-mini`
general	everything else	`gemini-2.0-flash` `llama-3.3-70b` `deepseek-chat` `gpt-4o` `claude-3-5-sonnet` `llama3.2 (Ollama)` `gemini-flash-lite via FreeLLMAPI`

Keyed · 11

Gemini, Groq, DeepSeek, OpenAI, Anthropic, OpenRouter, NVIDIA NIM, Mistral, Cohere, Cloudflare, Cerebras

Keyless · 2

Pollinations, OVHcloud anonymous tier — prompts go to third parties with no account.

Local · 1 (+1 legacy)

Ollama (`llama3.2`) · FreeLLMAPI gateway as last resort.

Aging risk

Model IDs are hard-coded 2024–25 snapshots. Retired IDs will fail until the table is edited.

Failover & cooldown

Inside one `chatWithTools` call — tools are never re-run by a failover, only the model request is retried elsewhere.

1 · Build queue
`getFailoverQueue(primary)` — primary first, then ready providers

→

2 · Try entry i
provider.chatWithTools(...)

→

3 · 429 / rate_limit?
`recordCooldown(provider, 75 s)`, move to i+1

→

4 · Success
returns `activeModel`, `isLowQuota`

What the user sees

- Timeline: *"Seamless Failover → model"* when the active model differs.
- Reply footer: *"Low Token Quota Alert"* when `isLowQuota`.
- All providers exhausted → `RateLimitCard` with a retry timer.

How "low quota" is decided

`isLowQuota = i ≥ queue.length - 2` — a position heuristic (running on one of the last two fallbacks). The "~10k tokens left" text is not measured.

Second safety net

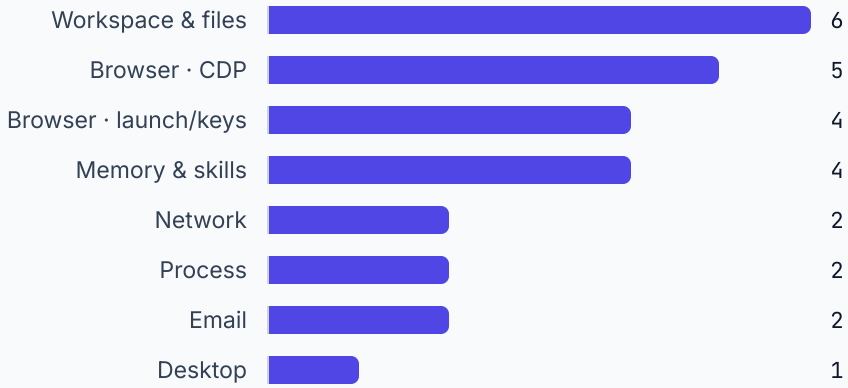
If the balancer throws, the adapter retries through the legacy FreeLLMAPI endpoint with its own bounded retry and thinking-block extraction.

ADR-003 rule kept: a model is pinned per profile; `auto` routing and failover are resilience, and every switch is shown — Jarvis never changes provider silently.

Tool catalog — 26 tools

Declared in `tool-registry.ts` (zod schema + JSON schema for the model), executed by one switch in `tool-executor.ts`.

Tools per domain



No generic shell tool. No mouse/keyboard control of arbitrary apps. Desktop vision is screenshot-only.

DOMAIN	TOOLS
Workspace & files	`workspace_list` `file_read` `project_inspect` `file_write_new` `file_patch` `create_pdf`
Browser · CDP	`browser_list_tabs` `browser_inspect_page` `browser_click_element` `browser_close_tab` `browser_scroll`
Browser · launch	`open_browser` `browser_shortcut` `browse_github` `list_chrome_profiles`
Memory & skills	`manage_user_memory` `list_skills` `load_skill` `suggest_skills`
Network	`web_search` (DuckDuckGo HTML) `download_file` (50 MB cap)
Process	`start_dev_server` `stop_process`
Email	`send_email_direct` (SMTP TLS) `send_gmail` (Chrome)
Desktop	`screen_capture`

Permission model — ADR-004

Six categories decide whether an action runs, asks, or is refused. Approvals are bound to the exact arguments.

CATEGORY	DEFINITION	DEFAULT
read_in	Read inside the selected workspace	**Allowed**
read_out	Read outside the workspace	**Approval (today: denied)**
write_in	Create/modify inside workspace	**New: allowed · overwrite/delete: approval**
write_out	Write outside workspace	**Approval**
process	Spawn processes, dev servers, scripts	**Approval**
network	Search, navigation, downloads	**Allowed (downloads to staging)**

Approval token

```
hash = HMAC-SHA256(sortedJSON(args), sessionSecret)
```

- 32-byte secret, new every launch.
- Expires after 5 minutes; single use.
- Any argument change → hash mismatch → refused.
- Deny evicts the request; no silent retry.

Also enforced

Real-path resolution against symlink/junction escape (SEC-006); executable and hidden-file creation needs approval (SEC-007); child processes spawn with `shell: false` (SEC-001).

Spec vs code

ADR-004 binds action type + workspace + args. Code hashes args only.

Approval flow

From a gated tool call to execution. The approved action runs outside the agent loop.



Meanwhile, the model…

gets *"gated — tell the user you are waiting"* as the tool result and the run ends. It never learns whether the action ran or what it produced.

Denied

Request evicted, task marked `cancelled`, chat records "Action Cancelled". The model cannot retry the same token.

Fail-open edge

If no approval handler is wired, `start_dev_server` skips the gate and runs (acceptance C).

Gating coverage today

What the spec (ADR-004 + README) expects for side-effecting tools, versus what the code does.

6
RISKY PATHS SPEC SAYS MUST ASK

1
ACTUALLY ASK TODAY

Only `file_write_new` and `start_dev_server` call the policy engine at all. Email tools are classed `network` (allowed).

TOOL / PATH	SIDE EFFECT	SPEC	TODAY
file_write_new (new)	create file in workspace/output	**allow + timeline**	✓ allowed; exec/hidden blocked
file_write_new overwrite	replace existing file	**approval**	✗ silent overwrite
file_patch	modify project file	**approval**	✗ no policy call (.bak kept)
start_dev_server	spawn process	**approval**	✓ asks · ✗ fails open w/o handler
stop_process	kill process	**own only**	✓ Jarvis-owned ids only
download_file	write to disk	**staging folder**	✓ output dir, 50 MB cap
send_email_direct	publish externally	**approval**	✗ sends immediately
send_gmail autoSend	publish externally	**approval**	✗ opens + clicks Send
browser_click / shortcut	act in logged-in browser	**unclassified**	✗ no policy call

Storage & state

Four stores, all local. Only provider keys are encrypted at rest.

SQLite · sql.js

`userData/jarvis.sqlite`

WASM SQLite held in memory, exported to disk via temp file + rename (SEC-010).

`conversations` `messages` `tasks` `approvals` `events` `settings`

Provider vault

`userData/provider_vault.json`

API keys encrypted with DPAPI (`safeStorage`), bound to the Windows user. Also holds hub state: enabled providers, local-server flag/port.

USER.md

`<workspace>/USER.md`

Plain Markdown: profile, dev defaults, standing rules. Injected into every system prompt; edited by `manage_user_memory`, the Settings app, and accepted LearningEngine suggestions.

Artifacts

`output/ · output/diagnostics/`

PDFs, downloads, screenshots, sanitized diagnostics bundles (Markdown + JSON, secrets redacted).

Agent memory, honestly

Long-term memory = one Markdown file. No embeddings, no search over past runs, no record of which strategies worked.

Path risk

`dbPath = process.cwd()/userData`. A packaged app started from autostart or another folder gets a different, empty database.

Privacy

`USER.md` is tracked in git and holds personal contact data. Chats and memory are not encrypted at rest.

Browser automation — three channels

Zero-dependency Chrome DevTools Protocol driver (`utils/cdp.ts`, 489 lines) plus launch and keystroke fallbacks.

1 · Launch

`launchBrowser()` runs `chrome.exe` with a chosen `--profile-directory`, optionally minimized/off-screen. Profiles discovered from Chrome's `Local State`. Rule: always ask which profile.

2 · CDP (deterministic)

Needs Chrome started with `--remote-debugging-port=9222`.

`GET /json/list` `/json/new` `/json/close` `WebSocket Runtime.evaluate`

Read page text, click by selector/text, scroll, close tabs, wait for Gmail compose DOM then click Send.

3 · Keystrokes (blind)

PowerShell `WScript.Shell.SendKeys` into the focused window: `Ctrl+Enter`, `Ctrl+W`, `Ctrl+Shift+T`. Used when CDP is not available. No verification of what received the keys.

send_gmail (autoSend)
compose URL with to/subject/body

→

CDP up?
yes: wait for compose DOM (15 s), DOM-click Send, close tab

→

CDP down
sleep 5.5 s → SendKeys Ctrl+Enter → sleep 2 s → Ctrl+W

Not there yet

No HTML-to-data extraction fallback, no crawler, no headless browser of its own (`playwright-research` is only a skill text).

Exposure

Port 9222 gives any local process full control of the signed-in browser while it is open.

Processes & dev servers

`ProcessManager` (246 lines) owns every child process Jarvis starts — and only those.

1. **Spawn** — `child_process.spawn`, `shell: false`, hidden window, working dir = selected project. No admin rights.
2. **Register** — `Map<id, {proc, state}>` with command, PID, cwd, status `starting | running | stopped | failed`.
3. **Observe** — rolling `recentLogs` buffer from stdout/stderr; port detection by log parsing + polling.
4. **Broadcast** — `onUpdate` → `process:updated` → `ProcessStrip` in the UI and preview in the Workspace Canvas.
5. **Stop** — by Jarvis id only: `taskkill /pid N /t /f` kills the whole tree. `stopAll()` on app exit.

Verified

- Unknown ids and foreign PIDs are refused (acceptance C).
- Dev-server start asks first with exact command + cwd.

Workspace Canvas

Split pane showing the running server (iframe), local PDFs (blob URL in `<object>`), web pages and code, with desktop/mobile emulation.

Missing

No general "run this command" tool, no test runner tool, no long-running job queue. Process state is lost on restart.

UI architecture — four surfaces

One React 19 renderer bundle; "The Quiet Instrument" / Parchment & Pine design system (ADR-005).

SURFACE	ENTRY	COMPOSITION
A · Mascot	`mascot/MascotApp.tsx` (450)	three.js scene, GLB owl rig, `MascotAnimationController` states: loading, idle, attention, thinking, working, needsApproval, completed, error, hidden; 60 fps cursor tracking
B · Quick chat	`quickchat/QuickChatApp.tsx` (535)	Compact drawer toggled from the mascot or tray; same chat IPC
C · Main window	`App.tsx` (636)	`TitleBar` · `Sidebar` · `ChatHeader` · `MessageList` · `Composer` (@ mentions) · `PlanCard` · `EvidenceRail` · `ApprovalDialog` · `ProcessStrip` · `WorkspaceCanvas`
D · Settings app	`settings/SettingsApp.tsx` (1,142)	Panels: Providers & keys (ping test, local server) · Models & balancer · Prompts & USER.md · Windows & behaviour

Design tokens

Light default + dark; text contrast ≥ 14:1 (WCAG AAA); fonts bundled locally; Windows 11 titlebar overlay recoloured with the theme.

Shared UI kit

`components/ui`: Button, Badge, Dialog (draggable, clamped to viewport), Field, Switch, Tooltip — on Radix primitives.

Performance

Composer keeps local state; message list memoized; static markdown plugins — fixed a 1 s typing lag.

Skills — runtime vs development

Two audiences, two folders. Runtime skills feed Jarvis's own small models; development skills feed coding agents.

Runtime skills in `skills/` — lines per SKILL.md



Decision (2026-11-09): replace long imported skills with short, purpose-built ones.

Runtime path

`SkillManager` scans `skills/` + `.agents/skills/` → keyword match → "Skill Advisor" hint → model calls `load_skill` → the full SKILL.md enters the context window.

Development path

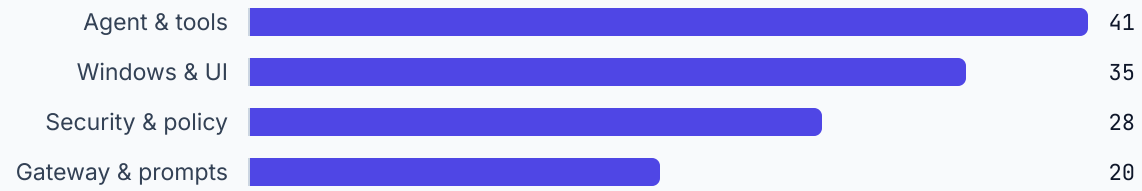
`.claude/skills/` (`api-from-docs`, `security-audit`, `status-deck`) — read by Claude Code directly and by Antigravity via `.agents/skills.json`, which also pulls 8 global engineering skills.

Kept apart on purpose: dev skills never load into Jarvis's runtime context.

Unit test suites

`npm test` — Vitest, 20 files, 124 tests, ≈10 s. All passing (verified 2026-11-09).

Tests per area



AREA	FILES (TESTS)
Agent & tools	tool-executor 13 · phase4-process 11 · cdp-browser 7 · agent-orchestrator 4 · pdf-generator 3 · user-memory 3
Windows & UI	canvas-document-resolver 9 · phase6-windows 6 · settings-window 6 · draggable-dialog 6 · mascot-animation 4 · owl-motion 4
Security & policy	policy-engine 8 · prompt-safety 7 · credential-vault 7 · security-remediation 6
Gateway & prompts	model-balancer 6 · prompt-engine 6 · model-adapter 5 · local-gateway-server 3

Other gates

- `npm run typecheck` — node + web projects, clean.
- `npm run security:scan` — 8/8 static checks.
- Playwright screenshot matrix + axe-core for UI v2.

What unit tests prove

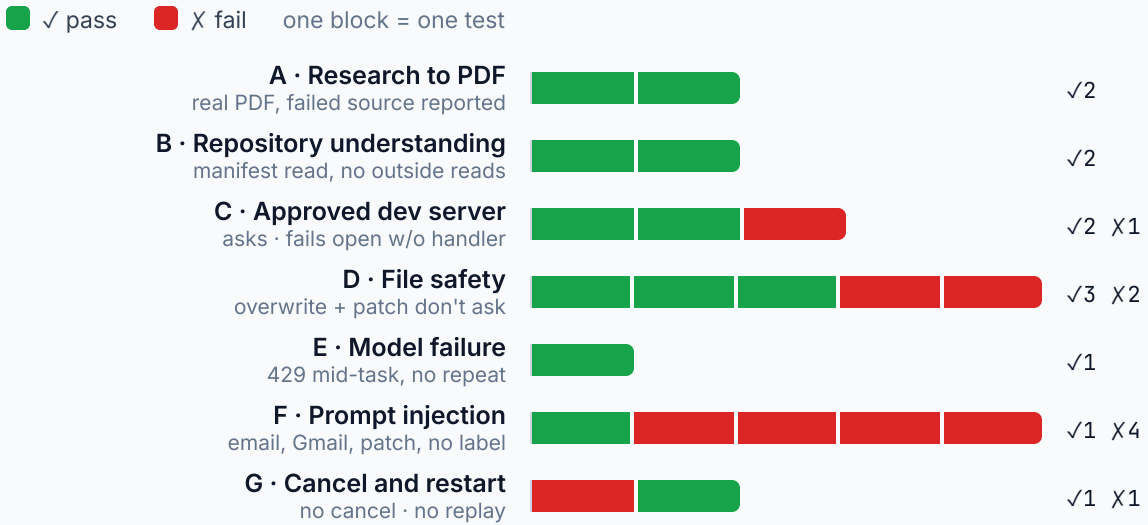
Each module behaves at its seam with neighbours mocked. They do not prove the assembled agent is safe or useful — that is the acceptance suite's job.

Blind spot

Tests live outside `tsconfig` includes, so they are not typechecked. A real-model ("live") tier does not exist yet.

MVP acceptance scoreboard — 12 / 20

`npm run test:acceptance` — plan §8 Tests A–G. Scripted (possibly hijacked) model; real orchestrator, executor, policy, PDF engine, files.



The 8 failures, by risk

- Hijacked model emails data out — SMTP and Gmail (F ×2).
- Hijacked model edits project files (F, D).
- Overwrite with `overwrite: true` needs no approval (D).
- `start_dev_server` runs if no approval handler (C).
- Tool output not marked untrusted (F).
- Cancel cannot stop the loop (G).

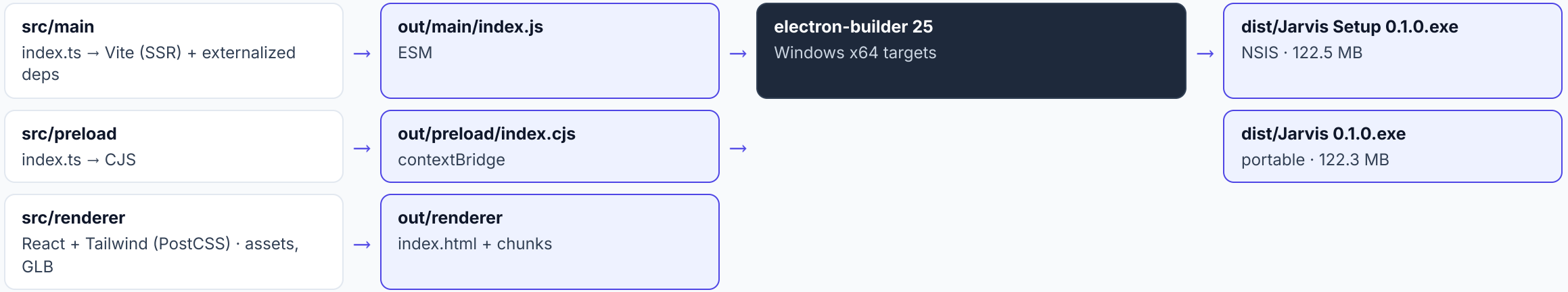
Design of the suite

Excluded from `npm test` on purpose: red here means "not MVP yet". SMTP, Chrome, CDP and keystrokes are mocked, so nothing leaves the machine.

VI · DELIVERY

Build, packaging, launch

electron-vite builds three bundles; electron-builder wraps them into Windows installers.



npm scripts

`dev · build · build:win`
`test · test:acceptance`
`typecheck · security:scan`
`config / settings (open settings app)`

Launch paths

`jarvis.bat` / `jarvis.sh` (start + auto-restart, PID file) · `Jarvis.lnk` desktop shortcut · tray icon · login item via `app.setLoginItemSettings` · CLI `--settings`.

Not yet

No code signing, no auto-updater, no CI pipeline (commits stay local by choice).

Known gaps & risks

Ranked by what they could cost. Every item is checked against the code, not taken from docs.

SEVERITY	GAP	WHERE / WHY IT MATTERS
Critical	**Email and file edits skip approval**	`tool-executor.ts`, `policy-engine.ts:127` — a prompt-injected page can make Jarvis send data out or rewrite code
High	**Gate fails open**	`start_dev_server` runs when no approval handler is set (`tool-executor.ts:976`)
High	**Local gateway: no auth, CORS ***	When enabled, any web page can use the stored API keys
High	**No cancellation**	Orchestrator has no abort signal; `task:cancel` only flips a flag
Medium	**Shallow agent**	5 turns, one model, no plan, no retry strategies, no memory of what worked
Medium	**Two execution paths**	`chat:send` (agent loop) and `task:submit-goal` (plan/steps) duplicate logic
Medium	**cwd-relative data paths**	`process.cwd()/userData` breaks for the packaged app
Medium	**Hard-coded model IDs · keyless providers**	Dated IDs will 404; anonymous endpoints receive prompts
Medium	**Concentrated code**	tool-executor / index.ts / SettingsApp ≈ 22% of all code
Low	**Tracker drift · personal data in git**	Ticket table stale; `USER.md` tracked; reply not streamed

Phases & what comes next

Phases 0–6 shipped. Phase 7 is active; the acceptance suite is the gate before expanding further.

P0	**Discovery & spikes**	Gateway validated; agent benchmarked; ADR-001…004
P1	**Secure foundation**	Electron shell, IPC, policy engine, DPAPI, SEC-001…011
P2	**Read-only agent loop**	Tools, function calling, PromptEngine
P3	**Documents & files**	PDF engine, safe writes, downloads
P4	**Coding workflow**	Dev servers, patcher, browser launch
P5	**Robustness & UI v2**	CDP driver, SMTP, USER.md, redesign
P6	**Windows & packaging**	Tray, autostart, diagnostics, installers
P7	**Native gateway & dogfooding** NOW	Multi-provider engine, settings app, canvas, acceptance harness

Next mission

1. **Close the 8 acceptance failures** — route every risky tool through the policy engine, fail closed, label untrusted content, add cancellation.
2. **Settle the vision** — autonomy level, learning loop, local model, multi-PC, Jarvis-hub (interview in progress).
3. **Reshape for scale** — architecture that survives those goals without a rewrite.

> *"Prove that one model can use one well-bounded tool reliably, then expand in small, verifiable steps."*